

AI 多模型智慧行政協作系統

文件 02 | Database Schema & Migration Specification

版本：v1.0

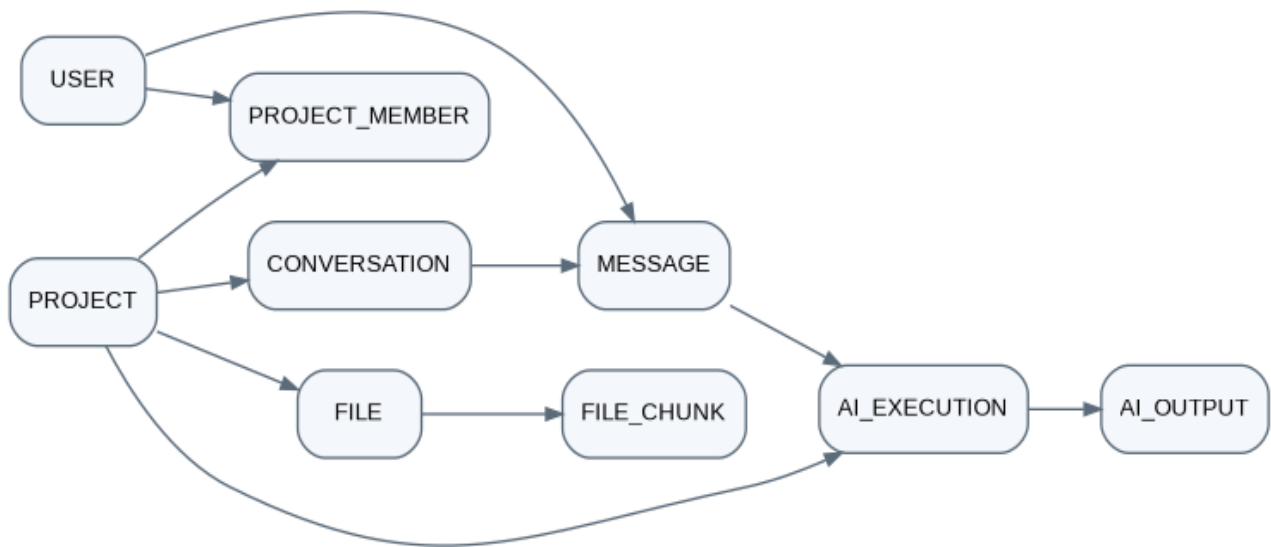
用途：工程開發、API 串接、資料庫設計、UI/UX 實作與驗收。

1. 資料層架構

採 PostgreSQL + pgvector + S3-compatible Object Storage。PostgreSQL 保存 transactional metadata；Object Storage 保存原始檔案；pgvector 保存 searchable chunks。

2. ER Diagram

圖 1：核心資料模型



Project 是資料隔離邊界；Conversation 屬於 Project；File/Chunk 屬於 Project/File；AI Execution 必須可追溯至觸發 Message。

3. PostgreSQL Schema

users / projects / project_members

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  display_name VARCHAR(255),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE projects (  
  id UUID PRIMARY KEY,  
  owner_id UUID NOT NULL REFERENCES users(id),  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  status VARCHAR(50) NOT NULL DEFAULT 'ACTIVE',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE project_members (  
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  role VARCHAR(30) CHECK (role IN ('OWNER', 'EDITOR', 'VIEWER')),  
  PRIMARY KEY (project_id, user_id)  
);
```

conversations / messages

```
CREATE TABLE conversations (  
  id UUID PRIMARY KEY,  
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,  
  title VARCHAR(255),  
  created_at TIMESTAMPTZ DEFAULT now(),  
  updated_at TIMESTAMPTZ DEFAULT now()  
);
```

```
);

CREATE TABLE messages (
  id UUID PRIMARY KEY,
  conversation_id UUID REFERENCES conversations(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id),
  role VARCHAR(30) NOT NULL,
  content TEXT NOT NULL,
  metadata JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

files / file_chunks

```
CREATE TABLE files (
  id UUID PRIMARY KEY,
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
  filename VARCHAR(500) NOT NULL,
  mime_type VARCHAR(100),
  storage_provider VARCHAR(50),
  storage_path TEXT NOT NULL,
  file_size BIGINT,
  checksum VARCHAR(128),
  processing_status VARCHAR(50) NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE file_chunks (
  id UUID PRIMARY KEY,
  file_id UUID REFERENCES files(id) ON DELETE CASCADE,
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
  chunk_index INTEGER NOT NULL,
  content TEXT NOT NULL,
  embedding VECTOR,
  metadata JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

ai_executions / ai_outputs

```
CREATE TABLE ai_executions (
  id UUID PRIMARY KEY,
  message_id UUID REFERENCES messages(id),
  project_id UUID REFERENCES projects(id),
  model VARCHAR(100) NOT NULL,
  task_type VARCHAR(100),
  task_complexity VARCHAR(30),
  routing_confidence DOUBLE PRECISION,
  input_tokens INTEGER,
  output_tokens INTEGER,
  latency_ms INTEGER,
  time_to_first_token_ms INTEGER,
  estimated_cost NUMERIC(12,6),
  status VARCHAR(30) NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE ai_outputs (
  id UUID PRIMARY KEY,
  execution_id UUID REFERENCES ai_executions(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  structured_output JSONB,
  validation_passed BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

4. Index / Isolation

```
CREATE INDEX idx_conversations_project
  ON conversations(project_id);

CREATE INDEX idx_messages_conversation
  ON messages(conversation_id, created_at);

CREATE INDEX idx_files_project
  ON files(project_id);

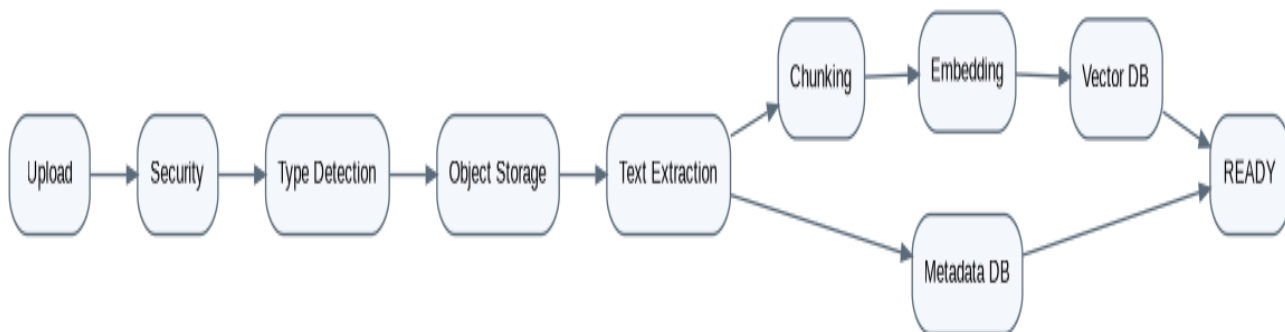
CREATE INDEX idx_file_chunks_project
  ON file_chunks(project_id);
```

```
CREATE INDEX idx_ai_executions_project_created
ON ai_executions(project_id, created_at DESC);
```

所有 retrieval query 必須以 current_project_id 為 filter；vector metadata 必須包含 project_id、file_id；不得跨 Project 搜尋。

5. File Processing Pipeline

圖 2：檔案處理 Pipeline



支援 PDF、DOCX、XLSX、PPTX、TXT、CSV、PNG、JPG、JPEG。原始檔保存於 Object Storage；抽取文字後 chunk、embedding，再進 pgvector。

6. Processing States

狀態	說明
UPLOADED	已上傳
PROCESSING	抽取 / chunk / embedding
READY	可被 retrieval 使用
FAILED	處理失敗
DELETED	已移除

7. Migration Strategy

使用 versioned migration，例如 001_init、002_vector、003_indexes、004_ai_metrics。Production schema change 必須可追蹤、可測試，避免 application startup 直接執行破壞性 schema change。

```
001_init.sql
002_enable_pgvector.sql
003_file_chunk_embedding.sql
004_indexes.sql
005_ai_execution_metrics.sql
```

附件：圖表名稱 + 原 UML / Mermaid

圖 1：核心資料模型

正文第 2 節原 UML / Mermaid：

```
erDiagram
    USER ||--o{ PROJECT_MEMBER : belongs_to
    PROJECT ||--o{ PROJECT_MEMBER : has
    PROJECT ||--o{ CONVERSATION : contains
    CONVERSATION ||--o{ MESSAGE : contains
    USER ||--o{ MESSAGE : sends
    PROJECT ||--o{ FILE : contains
    FILE ||--o{ FILE_CHUNK : contains
    PROJECT ||--o{ AI_EXECUTION : has
    MESSAGE ||--o{ AI_EXECUTION : triggers
    AI_EXECUTION ||--o{ AI_OUTPUT : produces
```

圖 2：檔案處理 Pipeline

正文第 5 節原 UML / Mermaid：

```
flowchart TD
    A["Upload File"] --> B["Security Check"]
    B --> C["Type Detection"]
    C --> D["Object Storage"]
    D --> E["Text Extraction"]
    E --> F["Chunking"]
    F --> G["Embedding"]
    G --> H["Vector DB"]
    E --> I["Metadata DB"]
    H --> J["READY"]
    I --> J
```